

Project 4 – EECS 152B

Cameron Peterson-Zopf

Module 1 – Noise Signal Generator

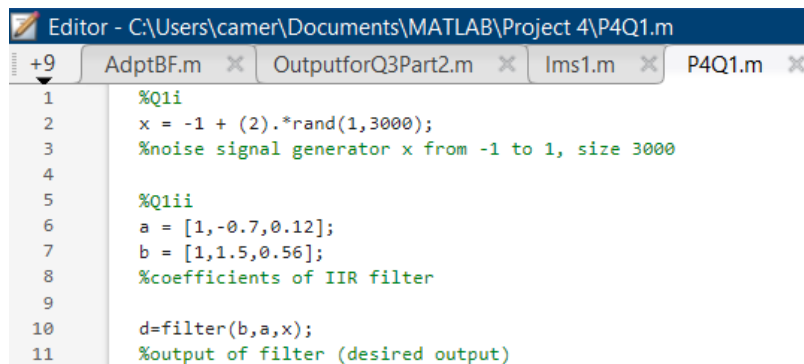
The first part of the project concerns designing an input for the system that will be built. The input will be a random noise generator which yields a sequence of uniformly distributed numbers with a length of 3000 over the interval of -1 to 1.

Next, an IIR system is developed that we wish to approximate with an FIR filter. The IIR filter is specified by the following second-order difference equation:

$$d(n) = 0.7d(n-1) - 0.12d(n-2) + x(n) + 1.5x(n-1) + 0.56x(n-2)$$

This IIR filter is implemented with the filter function in MATLAB. The filter function produces both the impulse response of the filter and the output of the filter with the input of the random noise.

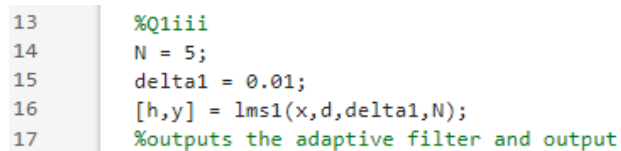
The MATLAB code for the above description is shown in figure 1.

The image shows a MATLAB Editor window with the title bar 'Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4Q1.m'. The window contains several tabs: 'AdptBF.m', 'OutputforQ3Part2.m', 'lms1.m', and 'P4Q1.m'. The 'P4Q1.m' tab is active, showing the following MATLAB code:

```
1 %Q1i
2 x = -1 + (2).*rand(1,3000);
3 %noise signal generator x from -1 to 1, size 3000
4
5 %Q1ii
6 a = [1,-0.7,0.12];
7 b = [1,1.5,0.56];
8 %coefficients of IIR filter
9
10 d=filter(b,a,x);
11 %output of filter (desired output)
```

Figure 1 – Code for IIR Filter and Random Noise Generator

Next comes the design of the adaptive filter. The adaptive filter will be an FIR filter of length N, whose coefficients are adjusted by the means of the LMS algorithm. The code for the filter is shown in figure 2 with filter length (N) = 5 and step size (delta1) = 0.01. These parameters can and will be adjusted as the project progresses.

The image shows a MATLAB Editor window with the title bar 'Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4Q1.m'. The window contains several tabs: 'AdptBF.m', 'OutputforQ3Part2.m', 'lms1.m', and 'P4Q1.m'. The 'P4Q1.m' tab is active, showing the following MATLAB code:

```
13 %Q1iii
14 N = 5;
15 delta1 = 0.01;
16 [h,y] = lms1(x,d,delta1,N);
17 %outputs the adaptive filter and output
```

Figure 2 – Code for FIR Adaptive Filter

Furthermore, the LMS algorithm can be unstable if the step size is too large; thus, we will monitor the convergence rate through computing a short-term average of the squared error. The formula for average square error (ASE) is as follows:

$$ASE(m) = \frac{1}{K} * \sum_{k=n+1}^{n+K} e^2(k)$$

The variable $m = n/K$ and $e(k)$ is the error function. Utilizing $K = 50$, $N = 5$, $\delta = 0.01$ and varying n in increments of 50, the average square error formula is plotted. The code for this is displayed in figure 3.

```

Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4Q1.m
+3 project1.m x P3P1Q1.m x P3P1Q3.m x P3P1Q3b.m x
1 %Q1i
2 x = -1 + (2).*rand(1,3000);
3 %noise signal generator x from -1 to 1, size 3000
4
5 %Q1ii
6 a = [1,-0.7,0.12];
7 b = [1,1.5,0.56];
8 %coefficients of IIR filter
9
10 d=filter(b,a,x);
11 %output of filter (desired output)
12 |
13 %Q1iii
14 N = 5;
15 delta1 = 0.01;
16 [h,y] = lms1(x,d,delta1,N);
17 %outputs the adaptive filter and output
18
19 %Q1a for N = 5, K = 10*N delta = 0.01
20 K = 10*N;
21 sqerr = zeros(1,K); %allocate memory for the output vector
22 SumSE = zeros(1,59); %allocate memory for the output vector
23 ASE = zeros(1,59); %allocate memory for the output vector
24
25 for n = 50:50:3000-K
26
27     %Calculate Square Error
28     for k=n+1:n+K
29         sqerr(k-n) = (d(k)-y(k)).^2;
30     end
31
32     %Calculate Sum of SE
33     temp = 0; %initialize
34     for p=1:K
35         temp = sqerr(p)+temp;
36     end
37     SumSE(n/K)=temp;
38
39     ASE(n/K)=(1/K).*SumSE(n/K);
40 end
41
42 m=1:59;
43 plot(m,ASE), xlabel('n/50 values'), ylabel('Avg Sq Error'),...
44 title ('Avg Sq Error for N = 5 & K = 50 & delta = 0.01');
grid

```

Figure 3 – Code for Plotting ASE

The plot generated from the code in figure 3 is shown in figure 4.

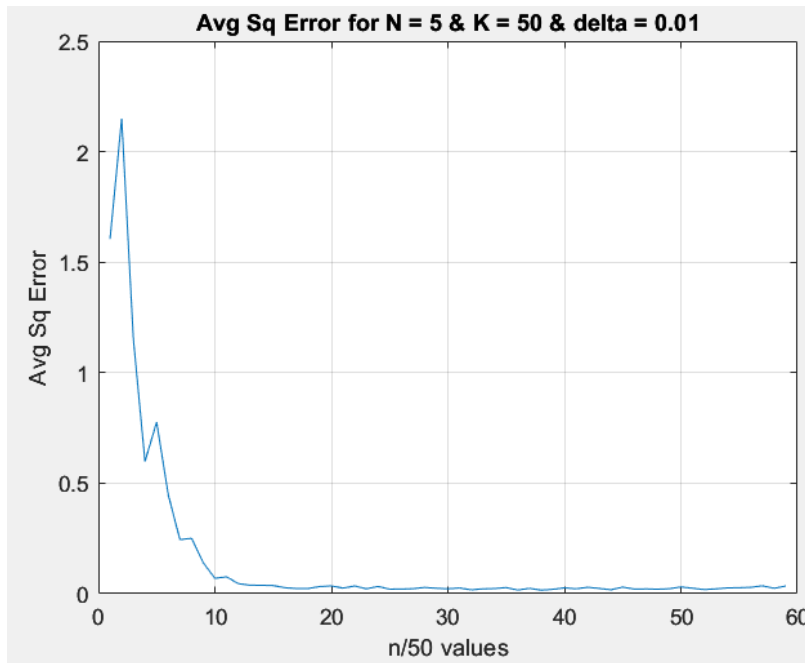


Figure 4 – ASE vs n/K for $K = 50$, $N = 5$, $\delta = 0.01$

The choice of step size is considered in the next section of the experiment. We begin with an FIR filter of length 5. Starting from 0.01 the step size is increased until 0.8. Here we see large deviations begin to develop. At 0.8 step size the ASE jumps at n/K values of 17 and 54 as shown in figure 5.

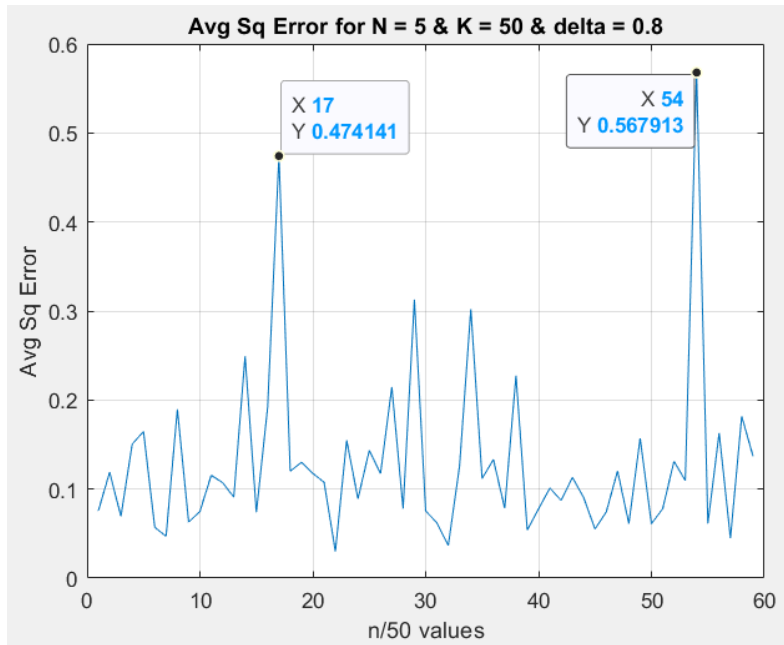


Figure 5 – ASE vs n/K for $K = 50$, $N = 5$, $\delta = 0.8$

Then, as δ is increased further to 0.85, the instability further deteriorates and undoubtedly diverges.

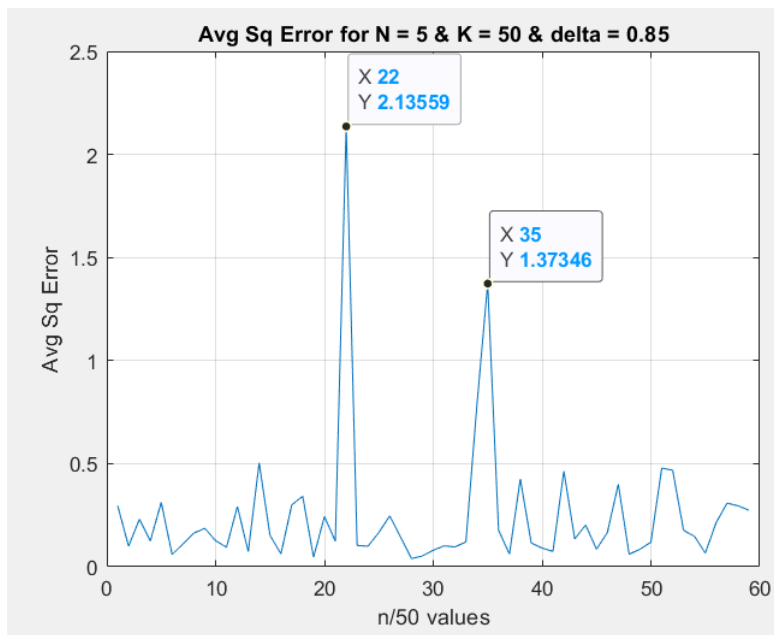


Figure 6 - ASE vs n/K for $K = 50$, $N = 5$, $\delta = 0.85$

This procedure is repeated for an FIR filter of length 10. Starting from 0.01 the step size is increased until 0.5. Here we see deviations begin to develop. At 0.5 step size the ASE jumps at the n/K value of 14 as shown in figure 7.

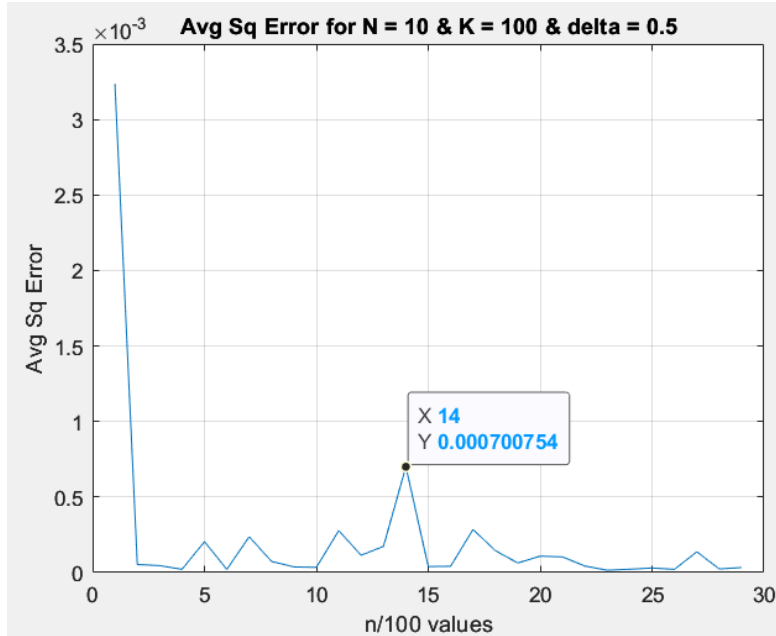


Figure 7 - ASE vs n/K for $K = 100$, $N = 10$, $\delta = 0.5$

Then, as the step size is increased to 0.55, the stability degrades further as shown in figure 8.

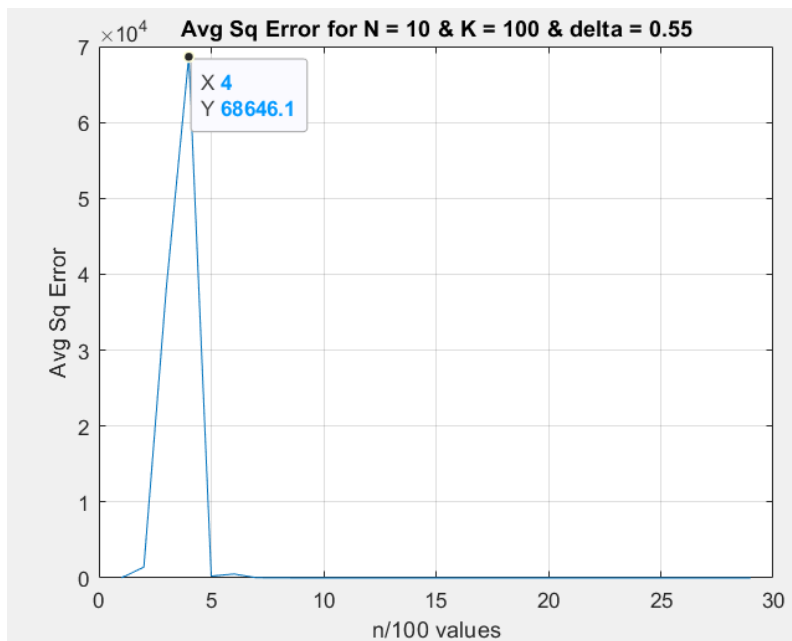


Figure 8 - ASE vs n/K for $K = 100$, $N = 10$, $\delta = 0.55$

Thus at $\delta = 0.55$, a much lower amount than the 0.85 for the 5 length filter, the 10 length FIR filter is unstable. The code developed for the FIR filter of length 10 is displayed in figure 9.

```

Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4stepsize.m
GetAng.m  P3P2Q1.m  P3P2Q1Retake.m  AdptBF.m  Outputf
1      %Q1i
2      x = -1 + (2).*rand(1,3000);
3      %noise signal generator x from -1 to 1, size 3000
4
5      %Q1ii
6      a = [1,-0.7,0.12];
7      b = [1,1.5,0.56];
8      %coefficients of IIR filter
9
10     d=filter(b,a,x);
11     %output of filter (desired output)
12
13     %Q1iii
14     N = 10;
15     delta1 = 0.55;
16     [h,y] = lms1(x,d,delta1,N);
17     %outputs the adaptive filter and output
18
19     %Q1a |
20     K = 10*N;
21     sqerr = zeros(1,K); %allocate memory for the output vector
22     SumSE = zeros(1,29); %allocate memory for the output vector
23     ASE = zeros(1,29); %allocate memory for the output vector
24
25     for n = 100:100:3000-K
26
27         %Calculate Square Error
28         for k=n+1:n+K
29             sqerr(k-n) = (d(k)-y(k)).^2;
30         end
31
32         %Calculate Sum of SE
33         temp = 0; %initialize
34         for p=1:K
35             temp = sqerr(p)+temp;
36         end
37         SumSE(n/K)=temp;
38
39
40     ASE(n/K)=(1/K).*SumSE(n/K);
41 end
42 m=1:29;
43 plot(m,ASE), xlabel('n/100 values'), ylabel('Avg Sq Error'),...
44     title ('Avg Sq Error for N = 10 & K = 100 & delta = 0.55');
45     grid

```

Figure 9 – Code for FIR Adaptive Filter of Length 10

Notice that the sizes of the variables and step size for n has changed due to the change in length.

The impulse responses of the adaptive filter and the IIR filter are compared in this section for adaptive filter lengths of 5 and 10. A discrete impulse response is used to generate the impulse response of the IIR filter. Figure 10 shows this code for N = 5.

```

Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4Pb.m
+4 OutputforQ3Part2.m x lms1.m x P4Q1.m x P4stepsize.m x P4Pb.m
1 %Impulse response of IIR filter
2
3 a = [1,-0.7,0.12];
4 b = [1,1.5,0.56];
5 %coefficients of IIR filter
6 x = zeros(1,10);
7 x(5)= 1;
8 hi=filter(b,a,x);
9 %output of filter (desired output)
10 ni = -4:1:5;
11 stem(ni,hi,'r');
12 hold
13 %Generate impulse of adaptive filter
14 xx = -1 + (2).*rand(1,3000);
15 %noise signal generator xx from -1 to 1, size 3000
16
17 d=filter(b,a,xx);
18 %output of filter (desired output)
19 %Q1iiii
20 N = 5;
21 delta1 = 0.01;
22 [hf,y] = lms1(xx,d,delta1,N);
23 %outputs the adaptive filter and output
24 n = 0:4;
25 stem(n,hf,'b'), xlabel('Discrete Time'), ylabel('Amplitude'),...
26 title ('Impulse Responses of IIR and FIR of N = 5');
27 grid

```

Figure 10 – Code for Impulse Response Comparison

The plot of the two impulse responses for N = 5 is displayed below in figure 11.

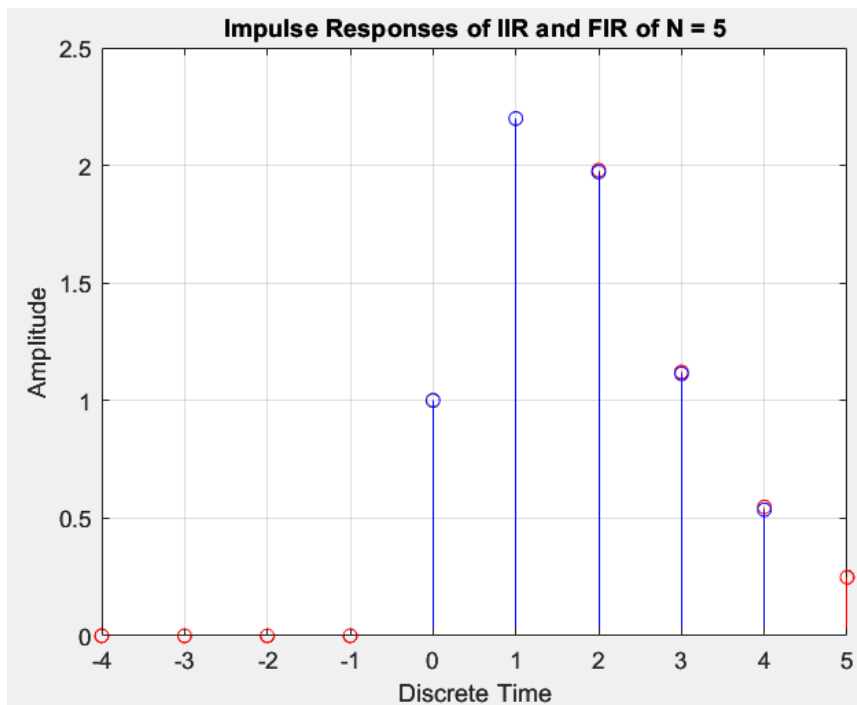


Figure 11 – Plot of Impulse Responses with FIR Filter of Length 5

As seen in figure 11 the FIR filter approximates the IIR filter exceedingly well, as the blue and red points are nearly on top of each other for the entire length of the FIR filter.

The plot of the two impulse responses for $N = 10$ is displayed below in figure 12.

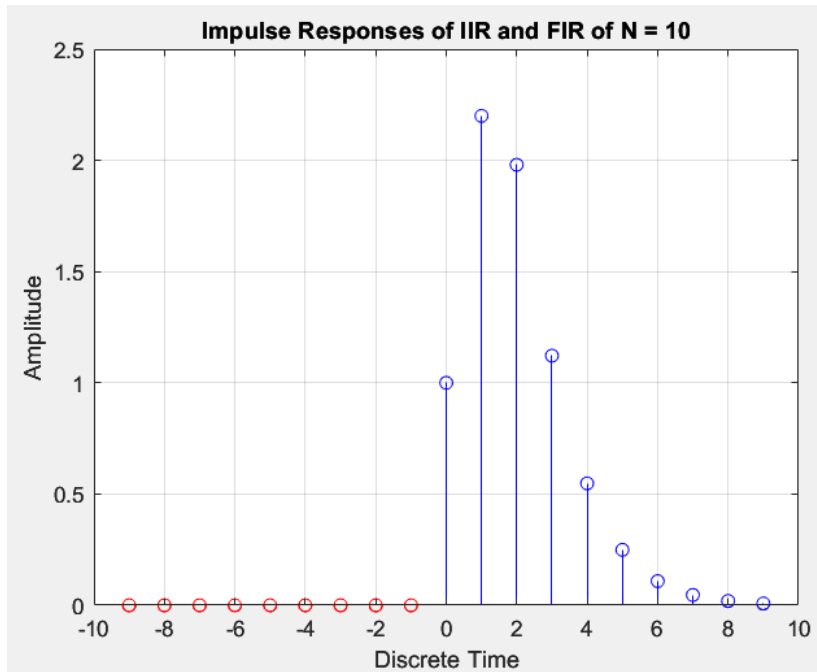


Figure 12 – Plot of Impulse Responses with FIR Filter of Length 10

As seen in figure 12, the adaptive filter approximates the IIR filter so well that the two graphs completely overlap for the entire length of the FIR filter. The FIR filter is causal and thus will begin at the zero point. The code for this plot is displayed below in figure 13.


```

Editor - C:\Users\camer\Documents\MATLAB\Project 4\P4Pb2.m
+5 | lms1.m x P4Q1.m x P4stepsize.m x P4Pb.m x P4Pb2.m x
1 | %impulse response of IIR filter
2 |
3 | a = [1,-0.7,0.12];
4 | b = [1,1.5,0.56];
5 | %coefficients of the IIR filter
6 | x = zeros(1,19);
7 | x(10)= 1;
8 | hi=filter(b,a,x);
9 | %output of filter (desired output)
10 | ni = -9:1:9;
11 | stem(ni,hi,'r');
12 | hold
13 | %Generate impulse of adaptive filter
14 | xx = -1 + (2).*rand(1,3000);
15 | %noise signal generator xx from -1 to 1, size 3000
16 |
17 | d=filter(b,a,xx);
18 | %output of filter (desired output)
19 | %Q1iii
20 | N = 10;
21 | delta1 = 0.01;
22 | [hf,y] = lms1(xx,d,delta1,N);
23 | %outputs the adaptive filter and output
24 | n = 0:1:9;
25 | stem(n,hf,'b'), xlabel('Discrete Time'), ylabel('Amplitude'),...
26 | title ('Impulse Responses of IIR and FIR of N = 10');
27 | grid

```

Figure 13 – Code for Impulse Responses with FIR Filter of Length 10

Lastly, the ASE and impulse functions are compared for different initialization values. First, we initialize everything to ones in the LMS algorithm as shown in figure 14.

```

Editor - C:\Users\camer\Documents\MATLAB\Project 4\lms1.m
+5 | lms1.m x P4Q1.m x P4stepsize.m x P4Pb.m x
1 | function [h,y] = lms1(x,d,delta1,N)
2 | % LMS Algorithm for Coefficient Adjustment
3 | % -----
4 | % [h,y] = lms(x,d,delta1,N)
5 | % h = estimated FIR filter
6 | % y = output array y(n)
7 | % x = input array x(n)
8 | % d = desired array d(n), length must be same as x
9 | %delta1 = step size
10 | % N = length of the FIR filter
11 | %
12 | M = length(x); y = zeros(1,M);
13 | h = ones(1,N);
14 | for n = N:M
15 |     x1 = x(n:-1:n-N+1);
16 |     y(n) = h*x1';
17 |     e = d(n)-y(n);
18 |     h = h + delta1*e*x1;
19 | end

```

Figure 14 – Code for Initialization of Ones

The ASE is repeated and plotted in figure 15.

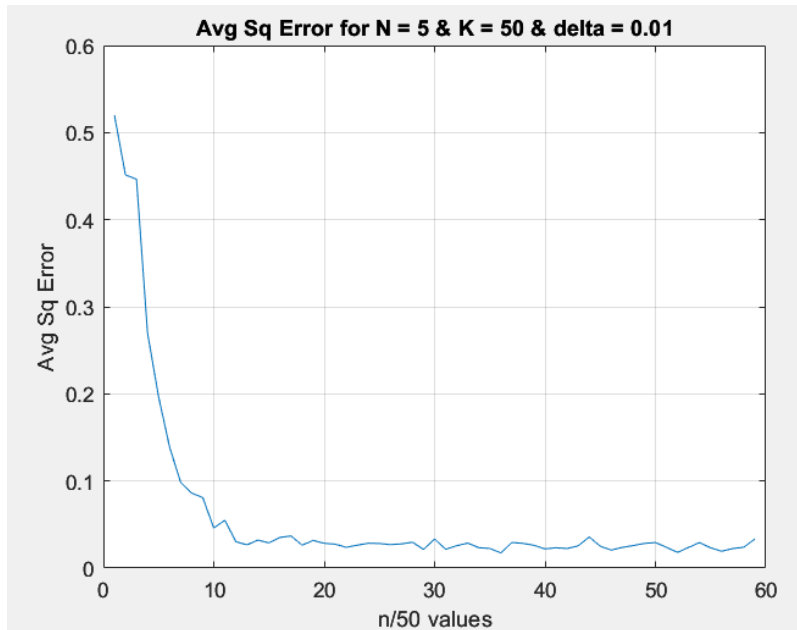


Figure 15 – ASE for Initialization of Ones

We see that when initialized as ones, the average square error begins at a lower value (hence better approximation as there is less error) and converges down at about the same rate. If we refer back to figure 4, we see that the error starts higher but quickly decreases down to where both are below 0.1 error by $m = 10$. Furthermore, the impulse functions still roughly match up for the initialization of ones as seen in figures 16 and 17.

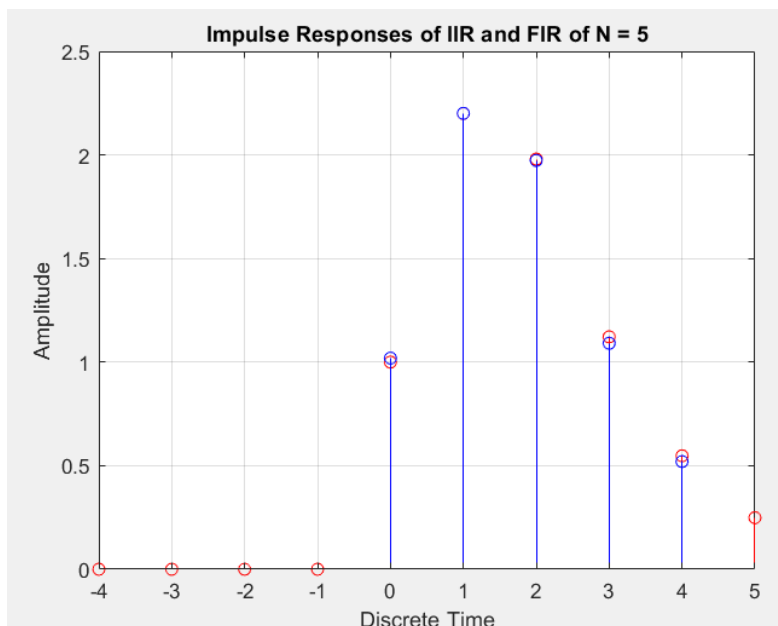


Figure 16 – Plot of Impulse Responses with FIR Filter of Length 5 w/ Initialization of Ones

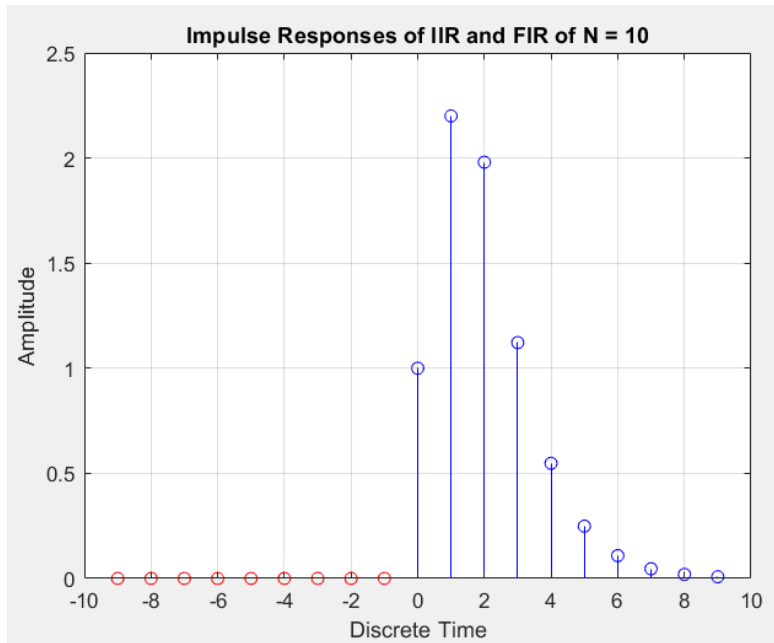


Figure 17 – Plot of Impulse Responses with FIR Filter of Length 10 w/ Initialization of Ones

In figure 16 we see that the impulse functions are slightly less matching but still reasonably close to one other. For length 10 there is no difference in initializations as shown in figure 17. In conclusion we see that by changing the initialization from zeros to ones there is some slight differences but overall, the adaptive filter is not that sensitive to changes in initialization.